

VULCAN

VULnerability Confirmation via Actual execution

A Benchmark for Execution-Verified AI-Generated Vulnerabilities
Research Design Specification

Eyad Hassan Abdelfatah

German International University (GIU), Cairo — Cybersecurity B.Sc.

Supervisors: Dr. Marwa Zamzam | Dr. Ahmed Maghawry

Working design document — revised after related-work survey

1. Purpose of this document

This is a living design specification, not a results chapter. It records the research questions, the measurement methodology, the exact scope, and the honesty boundaries that the benchmark must respect, so that the design can be reviewed and signed off before large-scale data collection begins. It supersedes earlier informal plans and reflects the related-work survey summarised in the Literature Review Sheet (notably CWEval, “Broken by Default,” the ASU application-level benchmark, and the iterative-degradation study), which narrowed and sharpened the contribution claimed here.

2. Thesis statement and research questions

Central question. What vulnerabilities do AI code-generation models produce when asked to implement real-world function tasks, how reliably can this be measured by *executing* the generated code rather than pattern-matching it, and how do execution failures and limited runtime feedback affect the observed results?

Primary research question (RQ1). Across models, CWEs, and tasks, at what rate does one-shot AI-generated code, when executed, cause a real dangerous operation (SQL execution, process execution, filesystem access) to run with attacker-shaped data reaching it?

Secondary research questions.

- **RQ2 (measurement validity).** What fraction of “could-not-evaluate” outcomes are caused by the model generating broken code, versus by the test harness being unable to construct the execution environment? (Answered via the ground-truth control, §6.)
- **RQ3 (execution-guided repair).** When the model is given only a runtime crash traceback and allowed a bounded number of repair attempts — with no security signal of any kind — does the observed vulnerability rate change relative to one-shot generation?
- **RQ4 (exploitability anchor).** For a curated subset, can the runtime-triggered signal be promoted to a full, end-to-end exploit through a real or realistically-constructed entry point?

3. Contribution and positioning (corrected)

Execution-based security evaluation of LLM code already exists (CWEval, Jan 2025) and rigorous exploitability confirmation already exists for memory-safety bugs in C/C++ (“Broken by Default”). The contribution of this thesis is therefore *not* “execution verification is new.” It is a specific, defensible *combination* that no single reviewed work provides:

1. Outcome-driven, execution-verified detection applied to **automatically-mined real-world library functions**, rather than to hand-authored self-contained tasks (CWEval’s 119 tasks) or synthetic prompts.
2. A **generic sink-interception oracle** that requires no per-task hand-authoring: the same instrumentation confirms a dangerous operation for *any* mined function, which is what makes the approach scale where hand-built oracles cannot.
3. A **ground-truth execution control**: each task’s real, human-written implementation is run through the identical harness, which lets “could-not-evaluate” be split into harness-caused and model-caused — a validity lever not present in the reviewed benchmarks.
4. **Functional injection-class** exploitation (CWE-89/78/22 and related), as opposed to memory-corruption formal proofs, with a curated set of fully-confirmed end-to-end exploits as anchors.

Relationship to application-level work. The ASU application-level benchmark deploys whole AI-generated web apps and attacks them over HTTP, achieving high ecological validity at small N. This thesis occupies the complementary point on the validity–scale trade-off: function granularity, a generic oracle, and a ground-truth control give higher statistical power and precise sink-level attribution, at the cost of assuming deployment-time attacker control for the automatic tier (addressed by Tier 2, §5).

4. Honesty boundary (non-negotiable)

The word “vulnerable” denotes a property of a *reachable path*, not of an isolated function. Accordingly:

- The automatic benchmark (Tier 1) reports “**the real dangerous sink executed with attacker-shaped data, under a stated attacker-controllability assumption**” — not “exploitable,” and not “full attacker-controlled exploit path.”
- The phrase “confirmed exploitable / full exploit path” is reserved strictly for Tier 2 cases, where a real or realistically-constructed entry point is demonstrated end to end, and for the six disclosed-CVE cases whose real-world attacker control was independently established by the original reporters.

5. Two-tier claim structure

Both tiers are built; they are layers of one pipeline, not alternatives.

Tier 1 — Automatic benchmark (large N). Every mined/generated candidate is executed in the instrumented harness. Output is a per-candidate verdict (§6) and aggregate rates with confidence intervals. This is the bulk of the quantitative result. Claim level: runtime-triggered sink execution under the attacker-controllability assumption.

Tier 2 — Exploit case studies (small, curated N). A fixed, small set (target 8–12) of already-triggered candidates is promoted to a full end-to-end exploit by supplying a real entry point (traced from the library’s own CLI/API surface) or a minimal, explicitly-labelled realistic caller, and demonstrating a concrete side effect. Claim level: confirmed exploitable. Tier 2 validates that the Tier 1 signal corresponds to real exploitation; four such demonstrations already exist (sqlite-utils SQLi, invoke command injection, tinydb path traversal, paramiko ProxyCommand).

“Manual” clarified. Tier 2 is bounded and short: each case is a small script of the kind already produced this project, not an open-ended effort and not hundreds of exploits.

6. Methodology

6.1 Mining (task source)

Real functions are mined from real open-source libraries and reduced to an “implement this” prompt (signature + docstring, body stripped), keeping only functions whose bodies reach a known CWE sink taxonomy. A *construction recipe* for the enclosing class is additionally mined from the repository’s own test suite (e.g. `Database(memory=True)` found in the project’s `confest.py`), restricted to calls whose arguments are literals or substitutable test fixtures, so the harness can build a real instance via the real constructor.

6.2 Instrumented execution harness (the oracle)

The harness patches the real standard-library sink implementations at their lowest common point (e.g. `subprocess.Popen.__init__`, `sqlite3` execution, `open`), plus a lower-level process-exec audit hook, and tracks a unique per-run marker by *value content* (surviving f-strings and formatting). A candidate is run in a disposable, resource-limited child process with a scratch working directory. Three honest outcomes are possible: `TRIGGERED`, `NOT_TRIGGERED`, `COULD_NOT_EXECUTE`. Stated limitations (reported, not hidden): same-kernel sandbox (not a security boundary against adversarial payloads); taint-by-value misses transformed inputs (a false-negative mode distinct from static analysis).

6.3 Generation (the subject under test)

For each mined prompt, the model receives the signature, docstring, and the enclosing class’s `__init__` as ordinary code context (never any security instruction, so default behaviour is measured). Sampling: k generations at temperature 0.7 for a rate-with-variance estimate, plus one temperature-0 deterministic reference for reproducibility.

6.4 RQ2 — the ground-truth classifier (“D”)

Each candidate’s real implementation is run through the identical harness as a control. Classification of a generated `COULD_NOT_EXECUTE`:

- real function executes, generated does not $\Rightarrow$ **model-caused** failure;
- real function also does not execute $\Rightarrow$ **infrastructure-caused** failure (excluded from the model’s denominator).

This automatic, control-based split is what makes the reported denominators defensible.

6.5 RQ2 — parameter construction recipes (“C”)

To shrink the infrastructure-caused bucket that D exposes, construction-recipe mining is extended beyond the enclosing class to *other* complex-typed parameters, mining safe constructor calls for them from the repository’s own tests. This improves both the control and the generated runs equally.

6.6 RQ3 — bounded execution-repair (“A”)

A second experimental arm. On a `COULD_NOT_EXECUTE` crash, the model is given **only the runtime traceback** and asked to make the code run without changing behaviour, for at most three attempts. The security verdict is never exposed to the model at any point (information

boundary below). This arm is reported separately from the one-shot baseline and additionally yields a vulnerability-transition analysis (one-shot vs repaired).

Information boundary (RQ3).

Into the repair loop: runtime exception + traceback *only*.

Never into the loop: sink result, CWE, taint verdict, any security signal.

Rationale: otherwise the experiment measures detector-evasion, not default behaviour.

7. Metrics

- **Execution rate** — fraction of generations that run (itself a reported model-quality metric, not a discarded precondition).
- **Trigger rate ($\approx \text{vuln}@k$)** — fraction of *executable* generations that trigger a sink, with the denominator stated explicitly and confidence intervals from the k samples.
- **Functional-vs-secure gap** — where functional test oracles are available, the drop from “runs correctly” to “runs correctly and does not trigger.”
- **Repair deltas (RQ3)** — change in execution rate and in trigger rate after bounded repair, plus the vulnerability-transition matrix (safe→vuln, vuln→safe, vuln→vuln, etc.).

8. Scope

CWEs. Core (built, validated): CWE-89 SQL Injection, CWE-78 OS Command Injection, CWE-22 Path Traversal. Planned additions, chosen because they fit the existing taint→single-sink architecture with only new sink patches (not new machinery): CWE-94/95 code injection (**eval/ exec**), CWE-502 insecure deserialization, CWE-918 SSRF, CWE-611 XXE. Deliberately excluded (require a different verification model): XSS, auth/ session logic, memory-safety.

Languages. Python is the deep, finished core (the instrumentation is Python-runtime-specific). Java is a stretch second language (requires a separately-built bytecode-instrumentation harness) if time permits. JavaScript is explicit future work. The gap statement claims only languages for which a harness is actually built.

Models. Local open-source first (Qwen2.5-Coder, DeepSeek-Coder via Ollama) for free, high-volume runs; then one GPT and one Claude model for the cross-model table. Exact model versions and access dates are pinned in the methodology for reproducibility.

9. Threats to validity

- **Construct.** “Sink triggered with marker data” is a proxy for “vulnerable.” Justified for Tier 1 under a stated assumption; anchored to real exploitation by Tier 2.
- **Internal.** Harness-caused failures could bias rates — mitigated by the ground-truth control (D) and construction recipes (C).
- **External.** Mined libraries and chosen models may not generalise — mitigated by multiple repositories, multiple models, and honest reporting of the corpus.
- **Reproducibility.** Stochastic generation — mitigated by k -sampling with CIs plus a temperature-0 reference, pinned versions, and released harness + corpus + raw generations.

10. Build order

1. Multi-sample generation layer over the already-mined prompts (the subject: AI code). (*next step*)
2. D: ground-truth-control classifier (model vs infrastructure).
3. C: parameter-level construction recipes (shrink infrastructure failures).
4. A: bounded execution-repair arm (RQ3) + transition analysis.
5. Scale across repositories, CWEs, and models; assemble Tier 2 exploit case studies; statistical analysis and write-up.

This design keeps vulnerability benchmarking as the centre of gravity: a scalable, honestly-scoped automatic measurement (Tier 1), validated by a small set of fully-confirmed exploits (Tier 2), with measurement validity (RQ2) and execution-guided repair (RQ3) as controlled secondary experiments. Full agentic / application-level generation is recorded as future work, not undertaken here.